

Problem A. Apparently Make UTPC

Let $N = |X| + |Y| + |Z|$. The key is to classify the two required intervals, called A and B , inside the final sequence C .

If one interval contains the other, it is enough to keep the larger interval. Then each remaining element can be put before or after the interval according to its comparison with the first element of the interval. Elements equal to that boundary value require only a constant number of cases, or can be tried both ways and the better lexicographic answer kept.

If the two intervals are disjoint, try which of A and B is placed on the left. Equivalently, one may take the lexicographically smaller order on the left. The elements that should go to the left of both intervals are processed by the containment case above, and the rest is handled similarly.

If the intervals overlap, again try which one starts first. Candidate overlap lengths can be checked with a suffix array and an LCP array. Once the overlap length is fixed, the instance reduces to the containment case. To choose the optimal overlap, maximize the number of elements that can be placed on the left, and among such choices minimize the resulting lexicographic sequence. This can be found by binary search plus suffix-array/LCP queries.

The entire solution can be implemented in $O(N)$ or $O(N \log N)$ depending on the suffix-array/LCP implementation.

Problem B. Binary Tree Counting

A direct DP is

$$\text{dp}(a, b, k) = \text{the number of possible subtrees whose preorder interval is } [a, a + k)$$

with inorder interval $[b, b + k)$. It has $O(N^3)$ states and $O(N)$ transitions, hence $O(N^4)$, which is too slow.

Use the classical bijections among binary trees, ordered trees, correct bracket sequences, and Dyck paths. Under the binary-tree to ordered-tree bijection, a vertex with preorder rank a and inorder rank b in the binary tree corresponds to a vertex whose preorder rank is a and whose postorder rank is b in the ordered tree. Under the ordered-tree to bracket-sequence bijection, these become a matching pair consisting of the a -th opening parenthesis and the b -th closing parenthesis. Thus the condition can be restated as a constrained Dyck-path counting problem.

After coordinate-compressing the specified pairs (A_i, B_i) , build the ordered tree that they force. If the forced structure is inconsistent, the answer is 0. For a node v , let d_v be the size of the diagonal segment corresponding to the matched pair. Define

$$f(v, d) = \text{number of valid path fragments inside } v \text{ when } d_v = d.$$

For the children $c_1, \dots, c_k$ of v , an inner DP

$$g_{v,d}(i, e)$$

chooses $d_{c_1}, \dots, d_{c_k}$, with $d_{c_i} = e$, and counts ways to travel through the forced points in order. The coefficients needed in transitions are numbers of monotone lattice paths that do not touch the line $y = x$; they are obtained by the reflection principle.

The naive transition appears to have five nested loops. The decisive optimization is that, for adjacent siblings c, c' , only

$$e \leq A_{c'} - A_c, \quad e' \leq B_{c'} - B_c$$

need to be tried. The sum of the products $(A_{c'} - A_c)(B_{c'} - B_c)$ over adjacent siblings is $O(N^2)$. This follows by converting the ordered tree back to a binary tree: it becomes the Cartesian tree of the sequence with indices B' and values A' , and the corresponding axis-aligned rectangles are disjoint except at endpoints.

Therefore the total number of effective (v, i, e, e') combinations is $O(N^2)$, and the remaining dimension d contributes a factor $O(N)$. The final complexity is $O(N^3)$.

Problem C. Convex Crusher

If all points are collinear, all N points can be chosen. Otherwise, consider a chosen subset and let l be a line containing the maximum number M of chosen points.

If $M \geq 4$, then $M + 1$ points are possible by adding one point off l . A set of size $M + 2$ can exist only by taking one point from each side of l , in such a way that the line through the two off-line points does not split the M points on l . A set of size $M + 3$ is impossible, because among three off-line points two must lie on the same side and force a convex quadrilateral with points on l .

If $M = 3$, four points are always possible. Five points are possible only in two special configurations: either one point is taken from each side and the three collinear points lie on the same side of their joining line, or two points on one side form a line through the middle collinear point. Six points are possible only in the “pinwheel” configuration described by these constraints; seven are impossible.

If $M = 2$, no three chosen points are collinear. Then four points must form a triangle with one interior point, and five points are impossible by the standard happy-ending argument.

Algorithmically, enumerate candidate lines. For the $M \geq 4$ case, angle sorting around endpoints of a line tests whether two off-line points can be added in $O(N^2 \log N)$, and an $O(N^3)$ implementation is also sufficient for the intended constraints. Then enumerate triples on a line to detect the $M = 3$ configurations, including the pinwheel case, in $O(N^3)$ or with bitsets in $O(N^3/w)$.

Problem D. Divisible by 11

Let n be the number of digits, c_i be the count of digit i , and s be the sum of all digits. By the divisibility rule for 11, after permuting the digits, the sum on odd positions and the sum on even positions must be equal modulo 11. Thus the sum on odd positions must be $s \cdot 2^{-1} \pmod{11}$.

First allow leading zeroes. If j_i copies of digit i are put on odd positions, the constraints are

$$\sum_{i=0}^9 j_i = \left\lceil \frac{n}{2} \right\rceil, \quad \sum_{i=0}^9 i j_i \equiv s \cdot 2^{-1} \pmod{11}.$$

For a fixed choice of j_i , the number of arrangements is

$$\left\lfloor \frac{n}{2} \right\rfloor! \left\lceil \frac{n}{2} \right\rceil! \prod_{i=0}^9 \frac{1}{j_i! (c_i - j_i)!}.$$

Therefore the needed coefficient is obtained from

$$\prod_{i=0}^9 (1 + xy^i)^{c_i} \pmod{1 - y^{11}},$$

multiplied by the factorial and inverse-factorial factors. The no-leading-zero answer is the leading-zero-allowed answer for the original multiset minus the same answer after removing one zero.

Since the modulus is not NTT-friendly, compute the generating function through a differential equation. For $g = \prod_i (1 + xy^i)^{c_i}$ and $g_i = 1 + xy^i$,

$$\left(\prod_i g_i \right) \frac{\partial g}{\partial x} = \left(\sum_i c_i g'_i \prod_{j \neq i} g_j \right) g.$$

This sparse differential equation determines the coefficients of g in increasing degree of x . To avoid multiplying polynomials in y repeatedly, evaluate y at the 11-th roots of unity modulo $2^{31} - 1$, solve the resulting one-variable equations, and recover the coefficients by an inverse DFT. The resulting complexity is linear in n up to small constants depending on the digit base and 11.

Problem E. Exchange or Not

Suppose the first $i - 1$ operations are fixed. If the current values at positions i and $i + 1$ are different, doing or not doing the i -th swap leads to different final sequences. If they are equal, the two choices are identical. Hence we may impose the canonical rule: do not swap when the two adjacent values are equal. Under this rule, operation sequences and final sequences are in one-to-one correspondence.

Now split the sequence at positions where the canonical operation does not swap. In each block, the first value must not appear again inside the same block; otherwise the forced canonical rule would break the block earlier. Thus the problem is to count valid ways to partition the array into intervals satisfying that condition.

Let $dp[i]$ be the number of ways to partition the prefix before position i . If an interval starts at i , it may end before the next occurrence of A_i . Add $dp[i]$ to all such ending positions. Range additions with prefix sums give an $O(N)$ solution. Equivalently, one can maintain dp by the current value and the total sum, which also yields $O(N)$.

Problem F. Finite Bracket Sequence

The answer is **Infinite** if and only if the substring X contains equally many opening and closing brackets.

If the counts are different, any candidate Y is either a substring of one copy of X , or has the form

$$\text{suffix of } X + X^k + \text{prefix of } X.$$

For large k , the net balance is nonzero and such a string cannot be a correct bracket sequence. Thus all valid Y have bounded length, and the answer is **Finite**.

If the counts are equal, use the cyclic-shift lemma for balanced bracket strings. Let A_i be the prefix balance of $X[1, i]$, and choose an index where A_i is minimum. The rotation $X[i + 1, |X|] + X[1, i]$ is a correct bracket sequence. Repeating this rotation any positive number of times gives arbitrarily long valid Y , so the answer is **Infinite**. Prefix sums answer all queries in $O(N + Q)$.

Problem G. Greatest Bracket Sequence

As in Problem F, a maximum exists exactly when the numbers of '(' and ')' in X are different. By reversing the whole string and swapping the roles of the brackets, it is enough to handle queries where X has positive total balance.

Let '(' be +1, ')' be -1, and let A_i be prefix sums. For a query $[L, R]$, put $d = A_R - A_L > 0$. A crucial observation is the following: if a correct bracket sequence Y is a substring of X^k for some k , but is not contained in one copy of X , then for some $L \leq l \leq r \leq R$,

$$Y = S[r, R] + S[L, l].$$

The proof chooses the smallest k for which Y appears; balance and nonnegative-prefix constraints force $k = 2$ and $l \leq r$.

For sequences contained in one copy, solve range queries by divide and conquer. For intervals crossing the midpoint, valid endpoints (l, r) are characterized by

$$A_l = \min_{l \leq t \leq m} A_t = A_r = \min_{m \leq t \leq r} A_t.$$

Fixing $t = A_m - A_l = A_m - A_r$, the best endpoints are the leftmost and rightmost feasible positions. The feasible range of t is an interval, and the objective is monotone, so binary search finds the optimum.

For sequences not contained in one copy, minimize $r - l$ under the symmetric prefix-min conditions

$$A_l = \min_{L \leq t \leq l} A_t, \quad A_r = \min_{r \leq t \leq R} A_t, \quad A_R - A_r = A_L - A_l.$$

The maximal feasible depth t is found by parallel binary search. Monotone stacks, processed in decreasing L and increasing R , answer the feasibility queries. This gives $O(N \log N + Q \log^2 N)$. A more refined implementation described in the official alternative editorial reduces this to $O((N + Q) \log N)$.

Problem H. Heyawake-like Problem

There is no solution for $N = 1$. For every $N \geq 2$, a constructive solution exists.

The official construction is case-based. One version separates even and odd N and uses periodic grid patterns; the displayed examples for $N = 8$ and $N = 9$ show that the top-left $(3k + 1) \times (3k + 1)$ part is already the answer for size k , so the pattern extends to all larger even and odd cases. An alternative construction treats $N = 2$, $N = 3$, and $N \geq 4$ separately, then repeats a fixed block pattern.

Thus the algorithm is simply to hard-code the small cases and output the appropriate periodic pattern for the parity of N . The verification is local: each added block preserves the same neighborhood constraints as in the base diagram, so the construction scales indefinitely.

Problem I. Inversion Graph

Let $G(P)$ be the inversion graph of a permutation P . If the maximum matching size of $G(P)$ is m , the length of the longest increasing subsequence is $N - m$. Let $c(d, m)$ be the number of permutations whose inversion graph is a tree with diameter d and maximum matching size m . Then

$$A_d = \sum_{m=1}^{\lfloor N/2 \rfloor} c(d, m)(N - m)^K.$$

For $N \geq 3$ and $d \geq 3$, the trees that appear are exactly caterpillar graphs. Let the diameter path be $v_0, v_1, \dots, v_d$, and let ℓ_i be the number of leaves attached to v_i for $1 \leq i \leq d - 1$. For each sequence $(\ell_1, \dots, \ell_{d-1})$, there are exactly two permutations that generate it: the values on the diameter path must alternate up-down or down-up, and then the leaves are forced.

Classify the internal diameter vertices into: a vertices with a leaf used in the matching, b adjacent pairs on the diameter used in the matching, and c unused vertices. They satisfy

$$a + 2b + c = d - 1, \quad a + b = m.$$

Summing the resulting choices gives

$$c(d, m) = 2 \binom{N - d}{2m - d} \binom{N - m - 2}{N - d - 1},$$

with out-of-range binomial coefficients treated as zero. Special cases $N = 2$ and $d = 2$ are handled directly.

For all d , transform

$$A_d = (N - d)[x^{d-2}] \sum_{m=1}^{\lfloor N/2 \rfloor} b_m(x + x^2)^{m-1},$$

where

$$b_m = \frac{2(N - m)^K(N - m - 2)!}{(N - 2m)!(m - 1)!}.$$

If $f(x) = \sum_{i=0}^{\lfloor N/2 \rfloor - 1} b_{i+1}x^i$, the required polynomial is $f(x + x^2)$. Since $x + x^2 = (x + 1/2)^2 - 1/4$, it can be computed by Taylor shift and polynomial composition. The overall complexity is $O(N \log N + N \log K)$.

Problem J. Jelly

Instead of always taking the maximum of the sweetness increment and the spiciness increment, imagine that we may freely choose either one. The optimal value is unchanged.

For foods where the chosen attribute does not change before and after eating that food, the contribution is 0. For the foods where the choice changes, exactly one of A_i and B_i contributes positively and the other negatively. Put $D_i = A_i - B_i$; then each such food contributes either $+D_i$ or $-D_i$, and the signs must alternate.

The necessary and sufficient condition for assigning contributions is that the number of $+D_i$ terms and $-D_i$ terms differ by at most one. Therefore, after sorting D_i , the positive terms should be taken from the largest values and the negative terms from the smallest values. Try the possible counts satisfying the difference condition and maximize the total. This is $O(N \log N)$. One can further show that at most one food needs contribution 0, but this is not needed for the sorted solution.

Problem K. Keep or Gamble

View the score as being earned immediately when a card is revealed, and reset to 0 when a cat card is revealed. Let

$$F = 2U + T,$$

which is the maximum score obtainable from all non-cat cards. Suppose the current score is X , there are R remaining cards, and C cat cards remain.

If we perform action A once, the expected increase from non-cat cards is $(F - X)/R$. The expected loss from cats is CX/R . Therefore action A is beneficial exactly when

$$X < \frac{F}{C+1}.$$

Once this inequality fails, it will never become true again if we continue optimally. Hence the optimal strategy is: continue action A while $X < F/(C+1)$, and stop with action B once $X \geq F/(C+1)$.

Except for the cases where a cat appears, the stopping score is either $\lceil F/(C+1) \rceil$ or one more. Compute the expected value of these cases and sum them. Panda cards can be ignored for this calculation, and iterating over the number of unicorn cards gives an $O(U + T + C)$ implementation.

Problem L. Linear Floor

Let $S_i = X_{i+1} - X_i$. A necessary condition is that S has at most two distinct values, and if there are two, they must be consecutive. By replacing X_i with $X_i - i \min S$, the problem reduces to either $S = \{0\}$ or $S = \{0, 1\}$. Shifting the intercept similarly reduces to $X_0 = 0$.

In the constant case $S = \{0\}$, the number of good triples with $M \leq M_0$ is a sum of linear functions. Thus a binary search over M , followed by reconstruction, yields the lexicographically K -th triple.

For a more direct tester-style solution, write $a = A/M$, $b = B/M$. The condition $\lfloor ak + b \rfloor = X_k$ is equivalent to

$$y_{\min}(a) \leq b < y_{\max}(a),$$

where

$$y_{\min}(a) = \max_k \{-ak + X_k\}, \quad y_{\max}(a) = \min_k \{-ak + X_k + 1\}.$$

Both are piecewise-linear envelopes and can be computed by convex-hull trick. The feasible region $y_{\min} < y_{\max}$ splits into at most four intervals where both boundaries are linear. On one interval $(a_l, a_r]$, if the two lines are $-k_1 a + X_1$ and $-k_2 a + X_2$, then for each M the valid A satisfy

$$\lfloor a_l M \rfloor \leq A < \lfloor a_r M \rfloor,$$

with a linear number of possible B . These sums are computable by floor-sum formulas. Binary searching M and then reconstructing A, B gives the answer.

The official supplement proves the “at most four intervals” fact: throughout the feasible range, the maximizing index of $y_{\min}$ comes from a set of size at most two, and the minimizing index of $y_{\max}$ from a set of size at most three, so the number of switches is at most three.

Problem M. Max Conference

Consider each maximal interval of question marks as a string segment bordered by fixed letters among A, B, and C. The objective is to minimize the number of adjacent equal pairs after filling all question marks.

For one segment, each letter has a maximum count that can be inserted without creating adjacency involving that letter. For a set of letters, the maximum count is the sum of individual maxima, capped by the number of question marks. If all relevant upper bounds are satisfied, the segment can be filled with no loss.

If not, the excess has the form that one letter, or later one pair of letters, is over its bound. Excess contributes a loss of twice the excess, except that the first loss in a segment may be only 1. Precompute, for each letter and segment, whether this first loss can be discounted.

For multiple segments, add the per-segment upper bounds and discount opportunities. First process the single-letter excess cases, prioritizing letters with larger excess and consuming discounts that are not useful elsewhere. If excess remains, it must be a two-letter-set excess; then use the available first-loss discounts greedily. The needed values can be maintained over all queries in $O(N + Q)$.

Problem N. Numerical Error

First, if $N \geq 21$, a solution always exists. If there are duplicate elements, this is immediate. Otherwise, take $M = 11$. The sums of reciprocals of any chosen 11 elements are less than

$$\sum_{k=1}^{11} \frac{1}{k} < 3.02.$$

For $N \geq 21$, the number of 11-element subsets is more than 3.5×10^5 . By the pigeonhole principle, two subsets must have indistinguishably close reciprocal sums, giving the desired equality under the required precision model.

Moreover, if any solution (X, Y) exists, then one exists with

$$M = \left\lfloor \frac{N}{2} \right\rfloor.$$

Remove common elements from $X \cap Y$, making them disjoint, and then add the same arbitrary unused elements to both sets until their size reaches $\lfloor N/2 \rfloor$.

Therefore, for implementation, replace N by $\min(N, 21)$, set $M = \lfloor N/2 \rfloor$, and search for two subsets of that size with the required equal value. Avoid ordinary floating-point comparisons where possible; exact rational arithmetic or carefully controlled high-precision arithmetic is safer.